

CÓMO TRABAJAR CON REPOSITORIOS EN GIT

Qué es un repositorio, comandos esenciales y herramientas de gestión

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Jaime Morales

Contenido

1. Qué es un repositorio	3
2. Git frente a SVN: un poco de historia	3
2.1. GitHub, GitLab y Bitbucket	3
3. Comandos principales de Git	4
3.1. Empezar y guardar cambios	4
3.2. Sincronizar con el remoto	4
3.3. Trabajar con ramas	4
3.4. Deshacer cambios	4
4. Herramientas para gestionar repositorios	5
5. Qué hemos aprendido	5

1. Qué es un repositorio

Cuando varias personas trabajan sobre el mismo código, tarde o temprano surge la necesidad de un lugar común donde guardarlo, compartirlo y llevar la cuenta de sus cambios. A eso se le llama **repositorio de código**: un espacio donde se almacena el software de un proyecto y que permite a distintos desarrolladores colaborar sobre él sin pisarse el trabajo.

Más allá de ser un simple almacén de archivos, un repositorio lleva la cuenta de cada modificación realizada. Esto es lo que se conoce como versionado: la posibilidad de volver atrás, comparar versiones o recuperar una parte concreta del código si algo sale mal.

★ Lo más importante

- ▶ Un repositorio almacena el código y, además, su historial completo de cambios.
- ▶ Permite el trabajo colaborativo y en paralelo de varias personas sobre el mismo proyecto.
- ▶ Aporta seguridad: copias de respaldo y posibilidad de recuperar versiones anteriores.

2. Git frente a SVN: un poco de historia

En el mundo de los repositorios de código conviven dos grandes filosofías: **Git**, con un control de versiones **distribuido**, y **SVN** (Subversion), con un control **centralizado**. La diferencia no es solo técnica, cambia por completo la forma de trabajar del equipo.

En Git, cada desarrollador tiene en su propio ordenador una copia completa del repositorio, historial incluido. Esto significa que se puede trabajar, consultar versiones antiguas o crear ramas sin necesidad de estar conectado a ningún servidor central; solo hace falta la red para sincronizar los cambios con los demás. SVN, en cambio, depende de un repositorio central: las copias de trabajo locales solo contienen la versión más reciente, y consultar el historial completo exige conectividad permanente.

Git fue creado por **Linus Torvalds** con el objetivo de ser rápido, eficiente y compatible con cualquier tipo de proyecto, y se distribuye bajo licencia pública general de GNU. Precisamente por su velocidad, sus copias completas del historial y su comunidad de herramientas, la mayoría de las grandes compañías han terminado adoptando Git como estándar, dejando a SVN un papel cada vez más residual.

★ Lo más importante

- ▶ Git es distribuido: cada copia local incluye el historial completo del proyecto.
- ▶ SVN es centralizado: el repositorio central concentra el historial y exige conectividad.
- ▶ Git es hoy el sistema de control de versiones más adoptado por la industria.

2.1. GitHub, GitLab y Bitbucket

Aunque Git es el motor, casi nunca se usa "a pelo": lo habitual es alojar los repositorios en una plataforma que añade una capa social y de gestión encima. **GitHub** es, con diferencia, la más popular: nació centrada en el código abierto y hoy pertenece a Microsoft, que la compró en 2018 por más de 7.500 millones de dólares sin que ello afectara a su papel central en la colaboración abierta.

GitLab se define a sí misma como una plataforma DevOps completa: no solo aloja el código, también integra la gestión de todo el ciclo de vida del software. Existe en dos sabores, una edición Community de código abierto y otra Enterprise de pago con funcionalidades adicionales.

Bitbucket, por su parte, tiene la particularidad de dar soporte tanto a Git como a Mercurial, y resulta atractiva para equipos pequeños gracias a sus repositorios privados gratuitos.

Las tres plataformas ofrecen funcionalidades muy similares (issues, pull/merge requests, integración continua); la elección suele depender más de la integración con el resto de herramientas del equipo que de diferencias técnicas.

3. Comandos principales de Git

Toda la potencia de Git se puede manejar desde la línea de comandos. Conviene distinguir entre los comandos que trabajan en local y los que sincronizan con el repositorio remoto.

3.1. Empezar y guardar cambios

```
git init // crea el esqueleto .git en la carpeta actual
git status // muestra qué cambios hay sin confirmar
git add archivo.js // empieza a trackear (o actualiza) un archivo
git commit -m "mensaje" // confirma los cambios trackeados
```

3.2. Sincronizar con el remoto

```
git fetch // descarga los cambios remotos sin fusionarlos
git merge nombre_rama // integra los cambios de una rama en la actual
git pull // equivale a fetch + merge en un solo paso
git push origin nombre_rama // sube la rama al servidor remoto
```

3.3. Trabajar con ramas

```
git branch // lista las ramas locales
git branch -a // lista ramas locales y remotas
git checkout -b nueva_rama // crea una rama y cambia a ella
git checkout -t origin/una_rama // crea una rama local que sigue a la remota
git branch -d nombre_rama // elimina una rama local
git remote prune origin // limpia referencias a ramas remotas ya borradas
```

3.4. Deshacer cambios

Cuando algo sale mal, Git ofrece dos formas muy distintas de "dar marcha atrás", y elegir la equivocada puede causar más de un dolor de cabeza. **git reset --hard HEAD** descarta los cambios que aún no se han confirmado; es

contundente y solo debería usarse sobre cambios que todavía no se han compartido con nadie. `git revert <hash_commit>`, en cambio, crea un nuevo commit que deshace uno anterior, conservando intacto el historial: es la opción segura cuando el commit problemático ya se ha subido al remoto.

★ Regla clave

- Nunca uses `reset --hard` sobre commits que ya has compartido con el equipo: reescribe el historial y puede provocar conflictos graves a tus compañeros. Para deshacer algo ya publicado, usa siempre `revert`.

4. Herramientas para gestionar repositorios

No todo el mundo se siente cómodo tecleando comandos, y por suerte no hace falta: existen clientes gráficos que permiten visualizar ramas, hacer commits o resolver conflictos con unos pocos clics.

SourceTree interactúa directamente con los repositorios Git y ofrece una interfaz muy visual para gestionar uno o varios repositorios a la vez. **GitKraken**, por su parte, es multiplataforma (Windows, macOS y Linux) y destaca por lo cuidada que resulta su interfaz de usuario, especialmente a la hora de visualizar el árbol de ramas de un proyecto complejo.

★ Lo más importante

- SourceTree y GitKraken evitan tener que usar la línea de comandos en el día a día.
- Ambas herramientas son gratuitas para uso personal y multiplataforma.
- Conocer los comandos de Git sigue siendo útil: algunas operaciones avanzadas solo están disponibles desde la terminal.

5. Qué hemos aprendido

A lo largo de este documento hemos recorrido los fundamentos para trabajar con repositorios de código: qué son, por qué Git se ha impuesto como estándar frente a SVN, dónde se pueden alojar (GitHub, GitLab, Bitbucket), qué comandos cubren el día a día de cualquier desarrollador y qué herramientas gráficas facilitan esa gestión cuando no apetece recurrir a la terminal.

Dominar el flujo básico —`init`, `add`, `commit`, `push`— y entender la diferencia entre sincronizar (`fetch/pull`) y deshacer (`reset/revert`) es la base sobre la que se apoya cualquier flujo de trabajo colaborativo más avanzado.

¡Enhorabuena! Documento completado